

RuMe - Design Rationale

Assignment 3: Adapter and Bridge together

Problem and agreed search rules

RuMe is a small console application for finding a roommate for shared renting or a tenant for an available room. A roommate must live in the requested city and have exactly the same planned monthly budget as the user. A tenant must live in that city and have a budget greater than or equal to the room price. Budgets are positive whole-number KZT amounts. City matching ignores letter case and surrounding spaces. All profiles are fictional, stored in memory, and used only for this assignment.

Bridge structure

HousingSearch is the Abstraction. It holds only a CandidateSource reference, validates search input, delegates city retrieval to that interface, and filters candidates using an overridable budget rule. RoommateSearch and TenantSearch are the two Refined Abstractions. CandidateSource is the Implementor. Its three production implementations are RuMeCandidateSource, UniversityCandidateSource, and LegacyCandidateSourceAdapter. The first uses a list, the second uses a city-indexed map, and the third wraps the incompatible component. Either search works with any of the three sources.

Genuine incompatibility and adaptation

LegacyHousingService is a deliberately separate educational component, not a real RuMe integration. Its fixed loadProfiles(int cityCode) method returns LegacyResult, containing an integer status and a String[][] matrix of ID, name, city code, and budget. It does not implement CandidateSource. In contrast, the common interface accepts a city name, returns List<Candidate>, and reports failures with CandidateSourceException. The adapter translates city names to codes, parses records into validated objects, and converts invalid-request, unavailable, and unknown statuses into common exceptions. Null responses, malformed records, invalid numbers, and recoverable runtime failures also become common exceptions without exposing native messages or causes. An unsupported city returns an empty list; successful empty results are not failures.

Why both patterns are needed

Bridge separates the changing search rules from the changing candidate catalogs, avoiding a search subclass for every catalog combination. It does not itself translate the legacy signature, data representation, or failure protocol. Adapter performs those translations, but an adapter alone would not separate the two independent variation axes. Here the adapter is one Implementor inside the same Bridge, rather than a separate demonstration.

Chosen complexity module: Dynamic implementor selection

AppRegistry uses the JDK ServiceLoader to discover providers declared in two META-INF/services configuration files. At runtime, the user enters a search code and a catalog code: rume, university, or legacy. Main asks the registry for that source and attaches it to the selected HousingSearch. Main contains no catalog-specific constructor calls or catalog-selection branches. The adapted source participates in exactly the same runtime selection.

Open/Closed Principle on both axes

To add a catalog, create a public CandidateSource implementation with a public no-argument constructor and a unique code. Add its class name to the CandidateSource provider file and its path to sources.txt. To add a search, create a public HousingSearch subclass implementing its four abstract methods and register it in the HousingSearch provider file and compiler list. These are configuration additions; existing Java classes, including Main and AppRegistry, remain unchanged. Test-only extra providers demonstrate independent extension without adding a fourth production catalog.

Tests and limitation

The repository contains 41 JUnit 5 test methods. Handwritten stubs verify delegation for both searches and failure translation by the real adapter. Additional tests cover all six search/source combinations, input validation, source contracts, runtime selection, and extension. See validation.md for the exact executed checks and the unexecuted JUnit status. One product limitation is that exact budget equality is a very strict roommate rule: a compatible person with a slightly different budget is excluded. The catalogs are demonstration data, not a production matching service.